

Case Studies II

Five more systems — each walked through the full framework from requirements to trade-offs.

60 Design a Distributed Rate Limiter — a shared counter at the edge

61 Design a Web Crawler — crawling the web politely at scale

62 Design Search Autocomplete — typeahead in milliseconds

63 Design Video Streaming (YouTube) — upload, transcode, and stream

64 Design Photo Sharing (Instagram) — media, feed, and the follow graph

Each case study reuses the framework and the patterns built across Parts I-IX.

CHAPTER 60

Design a Distributed Rate Limiter

Part X · Case Studies — a shared counter enforced at the edge.

WHAT THIS CASE STUDY SHOWS

- Enforcing a single limit across many servers.
- Choosing an algorithm and a shared counter store.
- Trading strict accuracy for latency and availability.

THE CORE CHALLENGE

Rate limiting one server is easy (Chapter 35). The hard part is enforcing **one global limit** across a fleet of gateway instances — without a shared counter, a client hitting different servers can exceed the limit many times over.

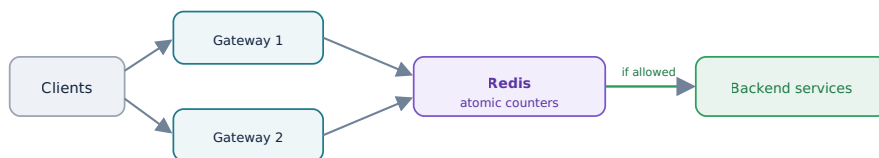
60.1 Requirements

Functional: allow up to N requests per window per key (user/IP/API key); reject the rest with **429**. **Non-functional:** very low added latency (it's on every request's hot path), highly available (it must not become a single point of failure), and accurate enough to protect the system.

60.2 Estimation

The limiter sits in front of *all* traffic, so it runs at full request volume — potentially hundreds of thousands of QPS. Every check must add well under a millisecond, which pushes the counter into **memory** (Chapter 8).

60.3 High-level design



every gateway checks and increments the same Redis counter — one global limit

60.4 Deep dive: the shared counter

KEY INSIGHT — A SHARED, ATOMIC COUNTER MAKES THE LIMIT GLOBAL

Each gateway, before forwarding a request, performs an **atomic increment-and-check** against a shared **Redis** counter keyed by the client and window (e.g. `user:123:minute`). Redis's single-threaded atomic operations prevent race conditions across gateways, so the limit holds globally. A **token bucket** or **sliding-window** algorithm (Chapter 35) lives in that

key. To keep the extra hop cheap, gateways can hold a small **local** allowance and reconcile with Redis periodically — trading a little accuracy for speed.

60.5 Trade-offs

The central counter is a potential bottleneck and dependency, so it must be replicated and fast; local pre-checks reduce load at the cost of slight over-admission. Decide **fail-open** (allow if Redis is down — availability first) vs **fail-closed** (reject — protection first). Perfect accuracy isn't the goal; **protecting the backend** with minimal latency is.

Key takeaways

- A **shared atomic counter** (Redis) turns per-server limiting into a **global** limit.
- Keep it on the hot path fast; use **local pre-checks** to cut load, accepting minor over-admission.
- Choose **fail-open vs fail-closed** deliberately; prioritize backend protection over exactness.

CHAPTER 61

Design a Web Crawler

Part X · Case Studies — crawling the web politely at massive scale.

WHAT THIS CASE STUDY SHOWS

- A large-scale producer/consumer pipeline built on a queue.
- De-duplication of URLs with a Bloom filter.
- Politeness, prioritization, and handling the web’s messiness.

THE CORE CHALLENGE

A crawler must fetch **billions** of pages, never get stuck in loops or re-fetch the same URL endlessly, and stay **polite** so it doesn’t hammer any single site. It’s a giant queue-driven pipeline with a hard de-duplication problem.

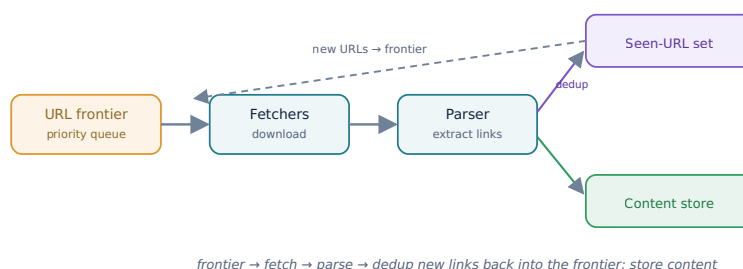
61.1 Requirements

Functional: start from seed URLs, fetch pages, extract links, and repeat; store page content; respect `robots.txt`. **Non-functional:** massive scale and throughput, politeness (per-domain rate limits), robustness to bad pages, and freshness (re-crawl over time).

61.2 Estimation

Billions of pages, tens of KB each → **petabytes** of content and enormous fetch bandwidth. The URL frontier holds billions of entries, so the “have I seen this URL?” check must be tiny and fast — a job for a probabilistic structure (Chapter 39).

61.3 High-level design



61.4 Deep dive: de-duplication and politeness

KEY INSIGHT — A BLOOM FILTER GUARDS THE FRONTIER

With billions of URLs, an exact “seen” set won’t fit in memory. A **Bloom filter** (Chapter 39) answers “have I seen this URL?” in tiny memory: “definitely not” → enqueue it; “probably yes” → skip (a rare false positive just means missing a page, which is acceptable). For **politeness**, the frontier is partitioned by **domain** with per-domain rate limits (Chapter 35)

so no site is overwhelmed, and `robots.txt` is honored. A priority queue favors important or fresh pages.

61.5 Trade-offs

The pipeline is a classic queue-based producer/consumer (Chapter 28), scaling horizontally by adding fetchers. Politeness caps throughput per domain (a courtesy vs speed trade-off). Traps (infinite calendars, spider traps), duplicate content, and re-crawl scheduling for freshness are the messy real-world concerns.

Key takeaways

- A crawler is a **queue-driven pipeline**: frontier → fetch → parse → enqueue new links.
- A **Bloom filter** makes billion-scale URL de-duplication fit in memory.
- Enforce **politeness** (per-domain limits, robots.txt) and prioritize for freshness.

CHAPTER 62

Design Search Autocomplete

Part X · Case Studies — *typeahead suggestions in milliseconds.*

WHAT THIS CASE STUDY SHOWS

- Serving suggestions with extreme low latency.
- Precomputing results with a trie and caching.
- Separating a fast read path from a slower analytics pipeline.

THE CORE CHALLENGE

Autocomplete fires on **every keystroke**, so each suggestion must return in a few milliseconds — far too fast to query a database live. The answer is to **precompute** the top suggestions for every prefix and serve them from memory.

62.1 Requirements

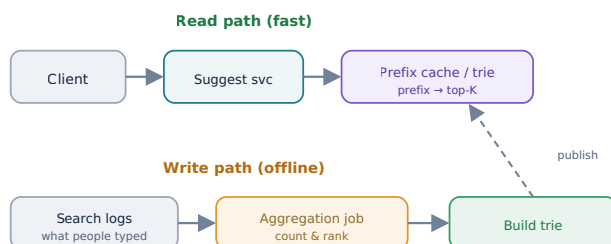
Functional: given a prefix, return the top-K most likely completions, ranked by popularity.

Non-functional: extremely low latency (a keystroke can't wait), high read volume, and suggestions that stay reasonably fresh.

62.2 Estimation

Every keystroke is a request, so read QPS is **enormous** (multiples of the search QPS). Writes — updating popularity from what people actually search — are comparatively rare and can lag. This read/write asymmetry defines the design (Chapter 53).

62.3 High-level design



62.4 Deep dive: the trie and precomputation

KEY INSIGHT — PRECOMPUTE TOP-K PER PREFIX; READ FROM MEMORY

Store completions in a **trie** (prefix tree), and at each node cache the **top-K** suggestions for that prefix so a lookup is a single fast traversal — no ranking at query time. The whole structure (or its hot part) lives in **memory/cache** for millisecond reads. Rankings come from an **offline pipeline** that aggregates real search logs, counts popularity, rebuilds the trie,

and publishes it periodically. This cleanly splits a blazing-fast **read path** from a slower, batch **write path** — the suggestions are slightly stale, which is fine.

62.5 Trade-offs

Precomputation trades **freshness** (suggestions update on a delay) for **speed**. The trie is sharded by prefix and replicated for read scale. Personalization, typo tolerance, and trending terms add complexity; most systems layer those on top of the popularity baseline.

Key takeaways

- Autocomplete must serve from **memory** — precompute **top-K per prefix** in a **trie**.
- Split a fast **read path** from an offline **aggregation/build** pipeline over search logs.
- Accept slightly **stale** rankings for millisecond latency.

CHAPTER 63

Design Video Streaming (YouTube)

Part X · Case Studies — upload, transcode, and stream at global scale.

WHAT THIS CASE STUDY SHOWS

- Asynchronous transcoding pipelines for uploaded media.
- Adaptive bitrate streaming delivered by a CDN.
- Separating heavy processing from fast delivery.

THE CORE CHALLENGE

Video is **huge** and must play smoothly on any device and network. Two problems dominate: processing each upload into many formats/resolutions (**transcoding**), and delivering enormous bandwidth to viewers worldwide with low startup latency (**the CDN**).

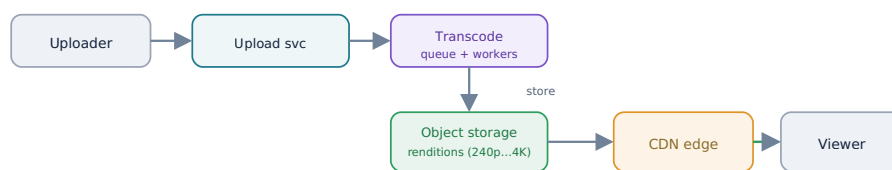
63.1 Requirements

Functional: upload a video; process it; stream it smoothly at multiple qualities; browse and search. **Non-functional:** massive storage and bandwidth, low playback startup and no buffering, and high availability. Reads (views) vastly exceed writes (uploads).

63.2 Estimation

Uploads are large (hundreds of MB to GB) and viewing generates **petabytes/day** of egress — bandwidth is the dominant cost (Chapter 49). Storage grows relentlessly. The view:upload ratio is thousands to one.

63.3 High-level design



upload → async transcode into many renditions → store → stream via CDN
heavy processing is offline; delivery is edge-cached

63.4 Deep dive: transcoding and adaptive streaming

KEY INSIGHT — PROCESS ONCE (ASYNC), DELIVER EVERYWHERE (CDN)

On upload, the raw file goes to object storage and a job is queued; **transcoding workers** (Chapter 28) convert it into multiple **resolutions and bitrates** (240p...4K), split into small segments — slow, CPU-heavy work kept entirely **off** the viewing path. For playback, **adaptive bitrate streaming** (HLS/DASH) lets the player switch quality segment-by-

segment based on the viewer's current bandwidth, avoiding buffering. All segments are served from a **CDN** (Chapter 8) close to viewers — essential for both latency and cost, since origin bandwidth would be ruinous.

63.5 Trade-offs

Storing many renditions multiplies storage but is required for smooth playback across devices; popular videos are widely cached, long-tail ones less so. Transcoding is expensive but asynchronous, so uploads return quickly (“processing...”). Metadata (titles, view counts, recommendations) is a separate, feed-like system (Chapter 57).

Key takeaways

- **Transcode asynchronously** into many renditions; keep heavy processing off the view path.
- Use **adaptive bitrate** (HLS/DASH) segments so playback adjusts to bandwidth.
- Deliver everything via **CDN** — the key to global latency and bandwidth cost.

CHAPTER 64

Design Photo Sharing (Instagram)

Part X · Case Studies — media, the feed, and the follow graph together.

WHAT THIS CASE STUDY SHOWS

- Combining media storage, a feed, and a social graph in one system.
- Processing images and serving them via a CDN.
- Reusing fan-out for the feed and sharding the graph.

THE CORE CHALLENGE

Photo sharing is a **synthesis** case: it fuses media handling (like YouTube, Chapter 63), a news feed (Chapter 57), and a follow graph. The interesting work is composing patterns you already know into one coherent system.

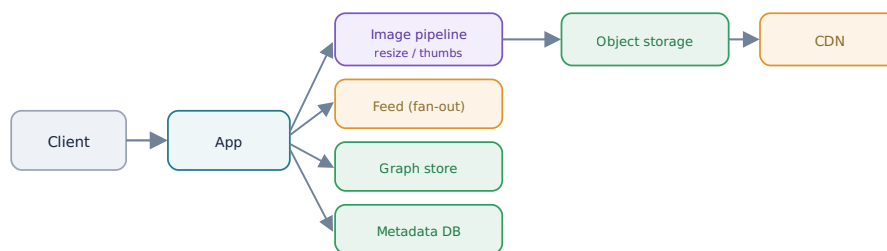
64.1 Requirements

Functional: upload a photo (with caption); follow users; view a home feed of followed users' photos; like/comment. **Non-functional:** low feed latency, fast image loads, read-heavy, high availability; eventual consistency for counts and feeds.

64.2 Estimation

Hundreds of millions of users; tens of millions of photo uploads/day at ~1 MB each → large but bounded storage, and heavy **image bandwidth** served by CDN. Feed reads dominate; the follow graph can be huge (Chapter 53).

64.3 High-level design



images processed & stored for CDN; feed uses fan-out; graph and metadata are separate stores

64.4 Deep dive: composing the pieces

KEY INSIGHT — IT'S THREE KNOWN SYSTEMS, WIRED TOGETHER

On upload, an **image pipeline** generates multiple sizes/thumbnails (like transcoding, Chapter 63), stores them in **object storage**, and serves them via **CDN** — only the small metadata (owner, caption, pointers) goes in the database (the metadata/blob split of Chapter 56). The **home feed** reuses the **fan-out** design (Chapter 57), including the celebrity hybrid.

The **follow graph** lives in its own store, **sharded** by user (Chapter 17), possibly a graph database (Chapter 15). Likes and comment counts are **eventually consistent** (Chapter 21). No single novel trick — the skill is choosing and connecting the right patterns.

64.5 Trade-offs

Precomputed feeds and multiple image sizes trade storage for fast reads (worth it, given read dominance). The graph's shard key matters for “who do I follow” queries and hotspots (celebrity accounts). CDN caching is central to both latency and cost.

Key takeaways

- Photo sharing **composes** media handling (Ch63), a fan-out feed (Ch57), and a sharded follow graph (Ch17).
- Process images into sizes, store as blobs, serve via **CDN**; keep only metadata in the DB.
- The senior skill on display is **pattern selection and composition**, not a single novel algorithm.

COMING NEXT — THE FINALE

Chapters 65–70 close the book with the last six case studies: ride-sharing (Uber), file storage & sync (Google Drive), ticket booking (Ticketmaster), a payment system, a nearby-places service, and a capstone that ties the whole method together.